# Liquid Time-Constant Networks

**Ramin Hasani[1,3*], Mathias Lechner[2*], Alexander Amini[1], Daniela Rus[1], Radu Grosu[3]**

[1] Massachusetts Institute of Technology (MIT) [2] Institute of Science and Technology Austria (IST Austria) [3] Technische Universität Wien (TU Wien)

*Authors with equal contributions

Published at: The Thirty-Fifth AAAI Conference on Artificial Intelligence (AAAI-21), pp. 7657–7666.

---

## Abstract

We introduce a new class of time-continuous recurrent neural network models. Instead of declaring a learning system's dynamics by implicit nonlinearities, we construct networks of linear first-order dynamical systems modulated via nonlinear interlinked gates. The resulting models represent dynamical systems with varying (i.e., liquid) time-constants coupled to their hidden state, with outputs being computed by numerical differential equation solvers. These neural networks exhibit stable and bounded behavior, yield superior expressivity within the family of neural ordinary differential equations, and give rise to improved performance on time-series prediction tasks. To demonstrate these properties, we first take a theoretical approach to find bounds over their dynamics, and compute their expressive power by the trajectory length measure in a latent trajectory space. We then conduct a series of time-series prediction experiments to manifest the approximation capability of Liquid Time-Constant Networks (LTCs) compared to classical and modern RNNs.

---

## 1. Introduction

Recurrent neural networks with continuous-time hidden states determined by ordinary differential equations (ODEs) are effective algorithms for modeling time series data that are ubiquitously used in medical, industrial and business settings. The state of a neural ODE, $\mathbf{x}(t) \in \mathbb{R}^D$, is defined by the solution of this equation (Chen et al. 2018):

$$\frac{d\mathbf{x}(t)}{dt} = f(\mathbf{x}(t), \mathbf{I}(t), t, \theta)$$

with a neural network $f$ parametrized by $\theta$. One can then compute the state using a numerical ODE solver, and train the network by performing reverse-mode automatic differentiation (Rumelhart, Hinton, and Williams 1986), either by gradient descent through the solver (Lechner et al. 2019), or by considering the solver as a black-box (Chen et al. 2018; Dupont, Doucet, and Teh 2019; Gholami, Keutzer, and Biros 2019) and apply the adjoint method (Pontryagin 2018). The open questions are: how expressive are neural ODEs in their current formalism, and can we improve their structure to enable better representation learning?

Rather than defining the derivatives of the hidden-state directly by a neural network $f$, one can determine a more stable continuous-time recurrent neural network (CT-RNN) by the following equation (Funahashi and Nakamura 1993):

$$\frac{d\mathbf{x}(t)}{dt} = -\frac{\mathbf{x}(t)}{\tau} + f(\mathbf{x}(t), \mathbf{I}(t), t, \theta)$$

in which the term $-\mathbf{x}(t)/\tau$ assists the autonomous system to reach an equilibrium state with a time-constant $\tau$. $\mathbf{x}(t)$ is the hidden state, $\mathbf{I}(t)$ is the input, $t$ represents time, and $f$ is parametrized by $\theta$.

### The LTC Formulation

We propose an alternative formulation: let the hidden state flow of a network be declared by a system of linear ODEs of the form $d\mathbf{x}(t)/dt = -\mathbf{x}(t)/\tau + \mathbf{S}(t)$, and let $\mathbf{S}(t) \in \mathbb{R}^M$ represent the following nonlinearity determined by $\mathbf{S}(t) = f(\mathbf{x}(t), \mathbf{I}(t), t, \theta)(A - \mathbf{x}(t))$, with parameters $\theta$ and $A$. Then, by plugging in $\mathbf{S}$ into the hidden states equation, we get:

$$\boxed{\frac{d\mathbf{x}(t)}{dt} = -\left[\frac{1}{\tau} + f(\mathbf{x}(t), \mathbf{I}(t), t, \theta)\right]\mathbf{x}(t) + f(\mathbf{x}(t), \mathbf{I}(t), t, \theta)A} \tag{1}$$

Eq. 1 manifests a novel time-continuous RNN instance with several features and benefits:

### Liquid Time-Constant

A neural network $f$ not only determines the derivative of the hidden state $\mathbf{x}(t)$, but also serves as an input-dependent varying time-constant ($\tau_{sys} = \frac{\tau}{1+\tau f(\mathbf{x}(t),\mathbf{I}(t),t,\theta)}$)

for the learning system. (Time constant is a parameter characterizing the speed and the coupling sensitivity of an ODE.) This property enables single elements of the hidden state to identify specialized dynamical systems for input features arriving at each time-point. We refer to these models as **liquid time-constant networks (LTCs)**. LTCs can be implemented by an arbitrary choice of ODE solvers. In Section 2, we introduce a practical fixed-step ODE solver that simultaneously enjoys the stability of the implicit Euler and the efficiency of the explicit Euler methods.

### Reverse-Mode Automatic Differentiation of LTCs

LTCs realize differentiable computational graphs. Similar to neural ODEs, they can be trained by variform gradient-based optimization algorithms. We settle to trade memory for numerical precision during a backward-pass by using a vanilla backpropagation through-time algorithm to optimize LTCs instead of an adjoint-based optimization method (Pontryagin 2018). In Section 3, we motivate this choice thoroughly.

### Bounded Dynamics — Stability

In Section 4, we show that the state and the time-constant of LTCs are bounded to a finite range. This property assures the stability of the output dynamics and is desirable when inputs to the system relentlessly increase.

### Superior Expressivity

In Section 5, we theoretically and quantitatively analyze the approximation capability of LTCs. We take a functional analysis approach to show the universality of LTCs. We then delve deeper into measuring their expressivity compared to other time-continuous models. We perform this by measuring the trajectory length of activations of networks in a latent trajectory representation. Trajectory length was introduced as a measure of expressivity of feed-forward deep neural networks (Raghu et al. 2017). We extend these criteria to the CT family.

### Time-Series Modeling

In Section 6, we conduct a series of eleven time-series prediction experiments and compare the performance of modern RNNs to the time-continuous models. We observe improved performance on a majority of cases achieved by LTCs.

### Why This Specific Formulation?

There are two primary justifications for the choice of this particular representation:

**I) Biological motivation.** The LTC model is loosely related to the computational models of neural dynamics in small species, put together with synaptic transmission mechanisms (Sarma et al. 2018; Gleeson et al. 2018; Hasani et al. 2020). The dynamics of non-spiking neurons' potential, $v(t)$, can be written as a system of linear ODEs of the form (Lapicque 1907; Koch and Segev 1998):

$$\frac{d\mathbf{v}}{dt} = -g_l\mathbf{v}(t) + \mathbf{S}(t)$$

where $\mathbf{S}$ is the sum of all synaptic inputs to the cell from presynaptic sources, and $g_l$ is a leakage conductance. All synaptic currents to the cell can be approximated in steady-state by the following nonlinearity (Koch and Segev 1998; Wicks, Roehrig, and Rankin 1996):

$$\mathbf{S}(t) = f(\mathbf{v}(t), \mathbf{I}(t)) \cdot (A - \mathbf{v}(t))$$

where $f(\cdot)$ is a sigmoidal nonlinearity depending on the state of all neurons $\mathbf{v}(t)$ which are presynaptic to the current cell, and external inputs to the cell $I(t)$. By plugging in these two equations, we obtain an equation similar to Eq. 1. LTCs are inspired by this foundation.

**II) Connection to Dynamic Causal Models.** Eq. 1 might resemble that of the famous Dynamic Causal Models (DCMs) (Friston, Harrison, and Penny 2003) with a Bilinear dynamical system approximation (Penny, Ghahramani, and Friston 2005). DCMs are formulated by taking a second-order approximation (Bilinear) of the dynamical system $d\mathbf{x}/dt = F(\mathbf{x}(t), \mathbf{I}(t), \theta)$, that would result in the following format (Friston, Harrison, and Penny 2003):

$$\frac{d\mathbf{x}}{dt} = (A + \mathbf{I}(t)B)\mathbf{x}(t) + C\mathbf{I}(t) \quad \text{with} \quad A = \frac{dF}{d\mathbf{x}},\ B = \frac{d^2F}{d\mathbf{x}\, d\mathbf{I}(t)},\ C = \frac{dF}{d\mathbf{I}(t)}$$

DCM and bilinear dynamical systems have shown promise in learning to capture complex fMRI time-series signals. LTCs are introduced as variants of continuous-time (CT) models that show great expressivity, stability, and performance in modeling time series.

---

## 2. LTCs Forward-Pass By Fused ODE Solvers

Solving Eq. 1 analytically is non-trivial due to the nonlinearity of the LTC semantics. The state of the system of ODEs, however, at any time point $T$, can be computed by a numerical ODE solver that simulates the system starting from a trajectory $x(0)$ to $x(T)$. An ODE solver breaks down the continuous simulation interval $[0, T]$ to a temporal discretization $[t_0, t_1, \ldots, t_n]$. As a result, a solver's step involves only the update of the neuronal states from $t_i$ to $t_{i+1}$.

LTCs' ODE realizes a system of stiff equations (Press et al. 2007). This type of ODE requires an exponential number of discretization steps when simulated with a Runge-Kutta (RK) based integrator. Consequently, ODE solvers based on RK, such as Dormand–Prince (default in torchdiffeq (Chen et al. 2018)), are not suitable for LTCs. Therefore, we design a new ODE solver that fuses the explicit and implicit Euler methods. Our discretization method results in greater stability, and numerically unrolls a given dynamical system of the form $dx/dt = f(x)$ by:

$$x(t_{i+1}) = x(t_i) + \Delta t f(x(t_i), x(t_{i+1})). \tag{2}$$

In particular, we replace only the $x(t_i)$ that occur **linearly** in $f$ by $x(t_{i+1})$. As a result, Eq. 2 can be solved for $x(t_{i+1})$ symbolically. Applying the Fused solver to the LTC representation, and solving it for $\mathbf{x}(t + \Delta t)$, we get:

$$\boxed{\mathbf{x}(t + \Delta t) = \frac{\mathbf{x}(t) + \Delta t\, f(\mathbf{x}(t), \mathbf{I}(t), t, \theta) A}{1 + \Delta t\big(1/\tau + f(\mathbf{x}(t), \mathbf{I}(t), t, \theta)\big)}} \tag{3}$$

Eq. 3 computes one update state for an LTC network. $f$ is assumed to have an arbitrary activation function (e.g., for a tanh nonlinearity $f = \tanh(\gamma_r x + \gamma I + \mu)$). The computational complexity of the algorithm for an input sequence of length $T$ is $O(L \times T)$, where $L$ is the number of discretization steps. Intuitively, a dense version of an LTC network with $N$ neurons, and a dense version of a long short-term memory (LSTM) (Hochreiter and Schmidhuber 1997) network with $N$ cells, would be of the same complexity.

**Algorithm 1: LTC update by fused ODE Solver**

**Parameters:** $\theta = \{\tau^{(N\times 1)} = \text{time-constant},\ \gamma^{(M\times N)} = \text{weights},\ \gamma_r^{(N\times N)} = \text{recurrent weights},\ \mu^{(N\times 1)} = \text{biases}\}$, $A^{(N\times 1)} =$ bias vector, $L =$ Number of unfolding steps, $\Delta t =$ step size, $N =$ Number of neurons

**Inputs:** $M$-dimensional Input $\mathbf{I}(t)$ of length $T$, $\mathbf{x}(0)$

**Output:** Next LTC neural state $\mathbf{x}_{t+\Delta t}$

**Function** `FusedStep`($\mathbf{x}(t)$, $\mathbf{I}(t)$, $\Delta t$, $\theta$):

$$\mathbf{x}(t + \Delta t)^{(N\times T)} = \frac{\mathbf{x}(t) + \Delta t\, f(\mathbf{x}(t), \mathbf{I}(t), t, \theta) \odot A}{1 + \Delta t\big(1/\tau + f(\mathbf{x}(t), \mathbf{I}(t), t, \theta)\big)}$$

▷ $f(\cdot)$, and all divisions are applied element-wise. $\odot$ is the Hadamard product.

**end Function**

$\mathbf{x}_{t+\Delta t} = \mathbf{x}(t)$

**for** $i = 1 \ldots L$ **do**

  $\mathbf{x}_{t+\Delta t} =$ `FusedStep`($\mathbf{x}(t)$, $\mathbf{I}(t)$, $\Delta t$, $\theta$)

**end for**

**return** $\mathbf{x}_{t+\Delta t}$

---

## 3. Training LTC Networks By BPTT

Neural ODEs were suggested to be trained by a constant memory cost for each layer in a neural network $f$ by applying the adjoint sensitivity method to perform reverse-mode automatic differentiation (Chen et al. 2018). The adjoint method, however, comes with numerical errors when running in reverse mode. This phenomenon happens because the adjoint method forgets the forward-time computational trajectories, which was repeatedly denoted by the community (Gholami, Keutzer, and Biros 2019; Zhuang et al. 2020).

On the contrary, direct backpropagation through time (BPTT) trades memory for accurate recovery of the forward-pass during the reverse mode integration (Zhuang et al. 2020). Thus, we set out to design a vanilla BPTT algorithm to maintain a highly accurate backward-pass integration through the solver. For this purpose, a given ODE solver's output (a vector of neural states) can be recursively folded to build an RNN and then apply Algorithm 2 to train the system. Algorithm 2 uses a vanilla stochastic gradient descent (SGD). One can substitute this with a more performant variant of the SGD, such as Adam (Kingma and Ba 2014), which we use in our experiments.

**Algorithm 2: Training LTC by BPTT**

**Inputs:** Dataset of traces $[I(t), y(t)]$ of length $T$, RNN-cell $= f(I, x)$

**Parameter:** Loss function $\mathcal{L}(\theta)$, initial parameters $\theta_0$, learning rate $\alpha$, Output weights $w = W_{\text{out}}$, bias $= b_{\text{out}}$

**for** $i = 1 \ldots$ number of training steps **do**

$(I_b, y_b) =$ Sample training batch

$x := x_{t_0} \sim p(x_{t_0})$

**for** $j = 1 \ldots T$ **do**

$x = f(I(t), x)$

$\hat{y}(t) = W_{\text{out}} \cdot x + b_{\text{out}}$

**end for**

$\mathcal{L}_{\text{total}} = \sum_{j=1}^{T} \mathcal{L}(y_j(t),\, \hat{y}_j(t))$

$\nabla \mathcal{L}(\theta) = \frac{\partial \mathcal{L}_{\text{total}}}{\partial \theta}$

$\theta = \theta - \alpha \cdot \nabla \mathcal{L}(\theta)$

**end for**

**return** $\theta$

**Complexity**

| | Vanilla BPTT | Adjoint |
|---|---|---|
| **Time** | $O(L \times T \times 2)$ | $O((L_f + L_b) \times T)$ |
| **Memory** | $O(L \times T)$ | $O(1)$ |
| **Depth** | $O(L)$ | $O(L_b)$ |
| **FWD accuracy** | High | High |
| **BWD accuracy** | High | Low |

**Table 1:** Complexity of the vanilla BPTT compared to the adjoint method, for a single layer neural network $f$. Note: $L =$ number of discretization steps, $L_f = L$ during forward-pass, $L_b = L$ during backward-pass, $T =$ length of sequence, Depth $=$ computational graph depth.

## 4. Bounds on $\tau$ and Neural State of LTCs

LTCs are represented by an ODE which varies its time-constant based on inputs. It is therefore important to see if LTCs stay stable for unbounded arriving inputs (Hasani et al. 2019; Lechner et al. 2020b). In this section, we prove that the time-constant and the state of LTC neurons are bounded to a finite range, as described in Theorems 1 and 2, respectively.

**Theorem 1 (Time-constant bounds)**

Let $x_i$ denote the state of a neuron $i$ within an LTC network identified by Eq. 1, and let neuron $i$ receive $M$ incoming connections. Then, the time-constant of the neuron, $\tau_{\text{sys}_i}$, is bounded to the following range:

$$\tau_i/(1+\tau_i W_i) \leq \tau_{\text{sys}_i} \leq \tau_i \tag{4}$$

The proof is constructed based on bounded, monotonically increasing sigmoidal nonlinearity for neural network $f$ and its replacement in the LTC network dynamics. A stable varying time-constant significantly enhances the expressivity of this form of time-continuous RNNs, as we discover more formally in Section 5.

**Theorem 2 (State bounds)**

Let $x_i$ denote the state of a neuron $i$ within an LTC, identified by Eq. 1, and let neuron $i$ receive $M$ incoming connections. Then, the hidden state of any neuron $i$, on a finite interval $\text{Int} \in [0, T]$, is bounded as follows:

$$\min(0, A_i^{\min}) \leq x_i(t) \leq \max(0, A_i^{\max}) \tag{5}$$

The proof is constructed based on the sign of the LTC's equation's compartments, and an approximation of the ODE model by an explicit Euler discretization. Theorem 2 illustrates a desired property of LTCs, namely **state stability** which guarantees that the outputs of LTCs never explode even if their inputs grow to infinity.

---

## 5. On The Expressive Power of LTCs

Understanding the impact of a NN's structural properties on their computable functions is known as the expressivity problem. The very early attempts on measuring expressivity of NNs include theoretical studies based on functional analysis. They show that NNs with three-layers can approximate any finite set of continuous mapping with any precision. This is known as the **universal approximation theorem** (Hornik, Stinchcombe, and White 1989; Funahashi 1989; Cybenko 1989). Universality was extended to standard RNNs (Funahashi 1989) and even continuous-time RNNs (Funahashi and Nakamura 1993). By careful considerations, we can also show that LTCs are also universal approximators.

**Theorem 3 (Universal approximation)**

Let $x \in \mathbb{R}^n$, $S \subset \mathbb{R}^n$ and $\dot{x} = F(x)$ be an autonomous ODE with $F : S \to \mathbb{R}^n$ a $C^1$-mapping on $S$. Let $D$ denote a compact subset of $S$ and assume that the simulation of the system is bounded in the interval $I = [0, T]$. Then, for a positive $\epsilon$, there exist an LTC network with $N$ hidden units, $n$ output units, and an output internal state $u(t)$, described by Eq. 1, such that for any rollout $\{x(t) \mid t \in I\}$ of the system with initial value $x(0) \in D$, and a proper network initialization:

$$\max_{t \in I} |x(t) - u(t)| < \epsilon \tag{6}$$

The proof defines an $n$-dimensional dynamical system and places it into a higher dimensional system. The second system is an LTC. The fundamental difference of the proof of LTC's universality to that of CT-RNNs (Funahashi and Nakamura 1993) lies in the distinction of the semantics of both systems where the LTC network contains a nonlinear input-dependent term in its time-constant module which makes parts of the proof non-trivial.

**Measuring Expressivity By Trajectory Length**

A measure of expressivity has to take into account what degrees of complexity a learning system can compute, given the network's capacity (depth, width, type, and weights configuration). A unifying expressivity measure of static deep networks is the **trajectory length** introduced in (Raghu et al. 2017). In this context, one evaluates how a deep model transforms a given input trajectory (e.g., a circular 2-dimensional input) into a more complex pattern, progressively.

We can then perform principal component analysis (PCA) over the obtained network's activations. Subsequently, we measure the length of the output trajectory in a 2-dimensional latent space, to uncover its relative complexity. The trajectory length is defined as the arc length of a given trajectory $I(t)$ (e.g., a circle in 2D space) (Raghu et al. 2017):

$$l(I(t)) = \int_t \left\| \frac{dI(t)}{dt} \right\| dt$$

By establishing a lower-bound for the growth of the trajectory length, one can set a barrier between networks of shallow and deep architectures, regardless of any assumptions on the network's weight configuration (Raghu et al. 2017).

**Computational Depth**

| Activations | Neural ODE | CT-RNN | LTC |
|---|---|---|---|
| tanh | 0.56 ± 0.016 | 4.13 ± 2.19 | 9.19 ± 2.92 |
| sigmoid | 0.56 ± 0.00 | 5.33 ± 3.76 | 7.00 ± 5.36 |
| ReLU | 1.29 ± 0.10 | 4.31 ± 2.05 | 56.9 ± 9.03 |
| Hard-tanh | 0.61 ± 0.02 | 4.05 ± 2.17 | 81.01 ± 10.05 |

**Table 2:** Computational depth of models. Note: # of tries = 100, input samples' $\Delta t = 0.01$, $T = 100$ sequence length, # of layers = 1, width = 100, $\sigma_w^2 = 2$, $\sigma_b^2 = 1$.

**Figure Descriptions**

**Figure 1: Trajectory's latent space becomes more complex as the input passes through hidden layers.** This figure illustrates how a simple 2D circular input trajectory ($x(t) = \sin(t),\ y(t) = \cos(t)$) is progressively transformed as it passes through a 6-layer deep network (width 100, tanh activations). Six panels show the PCA projection of each hidden layer's activations (L1 through L6) into a 2D latent space. The initial smooth circular trajectory becomes increasingly complex and convoluted with each successive layer, demonstrating how depth adds representational complexity. This visualization motivates the trajectory length measure for quantifying expressivity.

**Figure 2: Trajectory length deformation under various conditions.** This is a multi-panel figure showing PCA-projected 2D latent trajectories for Neural ODE (N-ODE), CT-RNN, and LTC models, with the computed trajectory length $l(\cdot)$ annotated for each. **(A)** Layers 1–5 with RK45 solver, tanh activations, depth 5, width 100: shows how trajectory complexity evolves across layers; LTC exhibits consistently longer trajectories. **(B)** Effect of weight distribution scaling ($\sigma_w^2 \in \{1, 2, 4\}$) with RK45, Hard-tanh, depth 1, width 100: LTC trajectory length grows faster-than-linearly with weight variance. **(C)** Effect of network width (100 vs 200) with RK45, ReLU, depth 1: trajectory length grows linearly with width for all models, but LTC maintains a large advantage (e.g., $l(\text{LTC}) = 266$ vs $l(\text{N-ODE}) = 81$ at width 100). **(D)** Width comparison (100 vs 200) with Hard-tanh: same trend as (C) but with even larger LTC advantage ($l(\text{LTC}) = 53859$ vs $l(\text{N-ODE}) = 138$ at width 100). **(E)** Three-layer network with Hard-tanh: shows trajectory deformation across layers 1–3; all models show exponential growth but LTC starts from a much higher baseline.

**Figure 3: Dependencies of the trajectory length measure.** Four-panel analysis figure. **(A)** Trajectory length vs. different ODE solvers (RK2(3), RK4(5), ABM1(13), TR-BDF2) — all variable-step solvers — with ReLU, depth 1, width 100: trajectory length is largely invariant to solver choice, confirming it is a property of the model not the numerics. **(B, top)** Trajectory length vs. network width ($k \in \{10, 25, 50, 100, 150, 200\}$) on log scale with tanh, depth 1, $\sigma_w^2 = 2$: demonstrates linear growth of trajectory length with width (curves appear linear on log-scale). **(B, bottom)** Variance-explained bar charts for the first 4 principal components across N-ODE, CT-RNN, and LTC, showing that LTC distributes variance more broadly across PCs (indicating higher-dimensional complexity). **(C)** Trajectory length vs. weight distribution variance ($\sigma_w^2 \in \{1, 2, 4, 8\}$) on log scale with ReLU: LTC shows super-linear growth. **(D)** Trajectory length vs. layers (L1–L6) with sigmoid, depth 6, width 100: trajectory length does not grow with depth for tanh/sigmoid continuous-time models, unlike static deep networks — a surprising finding.

**Theorem 4 (Trajectory Length Growth Bounds for Neural ODEs and CT-RNNs)**

Let $dx/dt = f_{n,k}(x(t), I(t), \theta)$ with $\theta = \{W, b\}$ represent a Neural ODE, and $\frac{dx(t)}{dt} = -\frac{x(t)}{\tau} + f_{n,k}(x(t), I(t), \theta)$ with $\theta = \{W, b, \tau\}$ a CT-RNN. $f$ is randomly weighted with Hard-tanh activations. Let $I(t)$ be a 2D input trajectory, with its progressive points (i.e., $I(t + \delta t)$) having a perpendicular component to $I(t)$ for all $\delta t$, with $L$ = number of solver-steps. Then, by defining the projection of the first two principal components' scores of the hidden states over each other as the 2D latent trajectory space of a layer $d$, $z^{(d)}(I(t)) = z^{(d)}(t)$, for Neural ODE and CT-RNNs respectively, we have:

**Neural ODE:**

$$\mathbb{E}\left[l(z^{(d)}(t))\right] \geq O\left(\frac{\sigma_w\sqrt{k}}{\sqrt{\sigma_w^2 + \sigma_b^2 + k\sqrt{\sigma_w^2 + \sigma_b^2}}}\right)^{d\times L} l(I(t)), \tag{7}$$

**CT-RNN:**

$$\mathbb{E}\left[l(z^{(d)}(t))\right] \geq O\left(\frac{(\sigma_w - \sigma_b)\sqrt{k}}{\sqrt{\sigma_w^2 + \sigma_b^2 + k\sqrt{\sigma_w^2 + \sigma_b^2}}}\right)^{d \times L} l(I(t)). \tag{8}$$

The proof follows similar steps as (Raghu et al. 2017) on the trajectory length bounds established for deep networks with piecewise linear activations, with careful considerations due to the continuous-time setup. The proof is constructed such that we formulate a recurrence between the norm of the hidden state gradient in layer $d+1$, $\|dz/dt^{(d+1)}\|$, in principal components domain, and the expectation of the norm of the right-hand-side of the differential equations of neural ODEs and CT-RNNs. We then roll back the recurrence to reach the inputs.

**Theorem 5 (Growth Rate of LTC's Trajectory Length)**

Let Eq. 1 determine an LTC with $\theta = \{W, b, \tau, A\}$. With the same conditions on $f$ and $I(t)$ as in Theorem 4, we have:

$$\mathbb{E}\left[l(z^{(d)}(t))\right] \geq O\left(\left(\frac{\sigma_w\sqrt{k}}{\sqrt{\sigma_w^2 + \sigma_b^2 + k\sqrt{\sigma_w^2 + \sigma_b^2}}}\right)^{d \times L} \times \left(\sigma_w + \frac{\|z^{(d)}\|}{\min(\delta t, L)}\right)\right) l(I(t)). \tag{9}$$

**Discussion of The Theoretical Bounds**

1. As expected, the bound for the Neural ODEs is very similar to that of an $n$ layer static deep network with the exception of the exponential dependencies to the number of solver-steps $L$.
2. The bound for CT-RNNs suggests their shorter trajectory length compared to neural ODEs, according to the base of the exponent. This results consistently matches the experiments presented in Figs. 2 and 3.
3. Figs. 2B and 3C show a faster-than-linear growth for LTC's trajectory length as a function of weight distribution variance. This is confirmed by LTC's lower bound shown in Eq. 9.
4. LTC's lower bound also depicts the linear growth of the trajectory length with the width $k$, which validates the results presented in Fig. 3B.
5. Given the computational depth of the models $L$ in Table 2 for Hard-tanh activations, the computed lower bound for neural ODEs, CT-RNNs and LTCs justify a longer trajectory length of LTC networks.

---

## 6. Experimental Evaluation

**Time Series Predictions**

We evaluated the performance of LTCs realized by the proposed Fused ODE solver against the state-of-the-art discretized RNNs, LSTMs (Hochreiter and Schmidhuber 1997), CT-RNNs (ODE-RNNs) (Funahashi and Nakamura 1993; Rubanova, Chen, and Duvenaud 2019), continuous-time gated recurrent units (CT-GRUs) (Mozer, Kazakov, and Lindsey 2017), and Neural ODEs constructed by a 4th order Runge-Kutta solver as suggested in (Chen et al. 2018), in a series of diverse real-life supervised learning tasks. The results are summarized in Table 3. We observed between 5% to 70% performance improvement achieved by the LTCs compared to other RNN models in four out of seven experiments and comparable performance in the other three.

| Dataset | Metric | LSTM | CT-RNN | Neural ODE | CT-GRU | **LTC (ours)** |
|---|---|---|---|---|---|---|
| Gesture | accuracy | 64.57% ± 0.59 | 59.01% ± 1.22 | 46.97% ± 3.03 | 68.31% ± 1.78 | **69.55% ± 1.13** |
| Occupancy | accuracy | 93.18% ± 1.66 | 94.54% ± 0.54 | 90.15% ± 1.71 | 91.44% ± 1.67 | **94.63% ± 0.17** |
| Activity recognition | accuracy | 95.85% ± 0.29 | 95.73% ± 0.47 | **97.26% ± 0.10** | 96.16% ± 0.39 | 95.67% ± 0.575 |
| Sequential MNIST | accuracy | **98.41% ± 0.12** | 96.73% ± 0.19 | 97.61% ± 0.14 | 98.27% ± 0.14 | 97.57% ± 0.18 |
| Traffic | squared error | 0.169 ± 0.004 | 0.224 ± 0.008 | 1.512 ± 0.179 | 0.389 ± 0.076 | **0.099 ± 0.0095** |
| Power | squared error | 0.628 ± 0.003 | 0.742 ± 0.005 | 1.254 ± 0.149 | **0.586 ± 0.003** | 0.642 ± 0.021 |
| Ozone | F1-score | 0.284 ± 0.025 | 0.236 ± 0.011 | 0.168 ± 0.006 | 0.260 ± 0.024 | **0.302 ± 0.0155** |

**Table 3:** Time series prediction. Mean and standard deviation, $n = 5$.

### Person Activity Dataset

We use the "Human Activity" dataset described in (Rubanova, Chen, and Duvenaud 2019) in two distinct frameworks. The dataset consists of 6554 sequences of activity of humans (e.g., lying, walking, sitting), with a period of 211 ms.

**1st Setting:** LTCs outperform all models and in particular CT-RNNs and neural ODEs with a large margin.

| Algorithm | Accuracy |
|---|---|
| LSTM | 83.59% ± 0.40 |
| CT-RNN | 81.54% ± 0.33 |
| Latent ODE | 76.48% ± 0.56 |
| CT-GRU | 85.27% ± 0.39 |
| **LTC (ours)** | **85.48% ± 0.40** |

**Table 4:** Person activity, 1st setting — $n = 5$.

**2nd Setting:** We carefully set up the experiment to match the modifications made by (Rubanova, Chen, and Duvenaud 2019) to obtain a fair comparison between LTCs and a more diverse set of RNN variants. LTCs show superior performance with a high margin compared to other models.

| Algorithm | Accuracy |
|---|---|
| RNN $\Delta t$* | 0.797 ± 0.003 |
| RNN-Decay* | 0.800 ± 0.010 |
| RNN GRU-D* | 0.806 ± 0.007 |
| RNN-VAE* | 0.343 ± 0.040 |
| Latent ODE (D enc.)* | 0.835 ± 0.010 |
| ODE-RNN* | 0.829 ± 0.016 |
| Latent ODE (C enc.)* | 0.846 ± 0.013 |
| **LTC (ours)** | **0.882 ± 0.005** |

**Table 5:** Person activity, 2nd setting, $n = 5$. Note: Accuracy for algorithms indicated by * are taken directly from (Rubanova, Chen, and Duvenaud 2019).

### Half-Cheetah Kinematic Modeling

We intended to evaluate how well continuous-time models can capture physical dynamics. We collected 25 rollouts of a pre-trained controller for the HalfCheetah-v2 gym environment (Brockman et al. 2016), generated by the MuJoCo physics engine (Todorov, Erez, and Tassa 2012). The task is then to fit the observation space time-series in an autoregressive fashion (Fig. 4). To increase the difficulty, we overwrite 5% of the actions by random actions.

**Figure 4: Half-cheetah physics simulation.** This figure shows a schematic of the HalfCheetah-v2 environment from OpenAI Gym. The cheetah robot is shown in profile view with labeled joint angles $\phi$. The task involves 17 input observations and 6 control outputs. Positive and negative directions of $\phi$ (joint angle) are indicated. A timeline axis (1–6) below shows the temporal progression of the simulation, with the cheetah in a running gait.

| Algorithm | MSE |
|---|---|
| LSTM | 2.500 ± 0.140 |
| CT-RNN | 2.838 ± 0.112 |
| Neural ODE | 3.805 ± 0.313 |
| CT-GRU | 3.014 ± 0.134 |
| **LTC (ours)** | **2.308 ± 0.015** |

**Table 6:** Sequence modeling — Half-Cheetah dynamics, $n = 5$.

## 7. Related Works

### Time-Continuous Models

TC networks have become unprecedentedly popular. This is due to the manifestation of several benefits such as adaptive computations, better continuous time-series modeling, memory, and parameter efficiency (Chen et al. 2018). A large number of alternative approaches have tried to improve and stabilize the adjoint method (Gholami, Keutzer, and Biros 2019), use neural ODEs in specific contexts (Rubanova, Chen, and Duvenaud 2019; Lechner et al. 2019) and to characterize them better (Dupont, Doucet, and Teh 2019; Durkan et al. 2019; Jia and Benson 2019; Hanshu et al. 2020; Holl, Koltun, and Thuerey 2020; Quaglino et al. 2020).

### Measures of Expressivity

Many works have tried to address why deeper networks and particular architectures perform well. Montufar et al. (2014) proposed counting the linear regions of NNs as a measure of expressivity. Eldan and Shamir (2016) showed that there exists a class of radial functions that smaller networks fail to produce. Poole et al. (2016) studied the exponential expressivity of NNs by transient chaos. Raghu et al. (2017) introduced trajectory length as a quantitative measure; we extended their analysis to CT networks.

---

## 8. Conclusions, Scope and Limitations

We introduced liquid time-constant networks. We showed that they could be implemented by arbitrary variable and fixed step ODE solvers, and be trained by backpropagation through time. We demonstrated their bounded and stable dynamics, superior expressivity, and superseding performance in supervised learning time-series prediction tasks, compared to standard and modern deep learning models.

### Long-term Dependencies

Similar to many variants of time-continuous models, LTCs express the vanishing gradient phenomenon (Pascanu, Mikolov, and Bengio 2013; Lechner and Hasani 2020), when trained by gradient descent. Although the model shows promise on a variety of time-series prediction tasks, they would *not* be the obvious choice for learning long-term dependencies in their current format.

### Choice of ODE Solver

Performance of time-continuous models is heavily tied to their numerical implementation approach (Hasani 2020). While LTCs perform well with advanced variable-step solvers and the Fused fixed-step solver introduced here, their performance is majorly influenced when off-the-shelf explicit Euler methods are used.

### Time and Memory

Neural ODEs are remarkably fast compared to more sophisticated models such as LTCs. Nonetheless, they lack expressivity. Our proposed model, in their current format, significantly enhances the expressive power of TC models at the expense of elevated time and memory complexity which must be investigated in the future.

### Causality

Models described by ODE semantics inherently possess causal structures (Schölkopf 2019), especially models that are equipped with recurrent mechanisms to map past experiences to next-step predictions. Studying causality of performant recurrent models such as LTCs would be an exciting future research direction as their semantics resemble DCMs (Friston, Harrison, and Penny 2003) with a bilinear dynamical system approximation (Penny, Ghahramani, and Friston 2005). Accordingly, a natural application domain would be the control of robots in continuous-time observation and action spaces where causal structures such as LTCs can help improve reasoning (Lechner et al. 2020a).

---

## Acknowledgments

R.H. and D.R. are partially supported by Boeing. R.H. and R.G. were partially supported by the Horizon-2020 ECSEL Project grant No. 783163 (iDev40). M.L. was supported in part by the Austrian Science Fund (FWF) under grant Z211-N23 (Wittgenstein Award). A.A. is supported by the National Science Foundation (NSF) Graduate Research Fellowship Program. This research work is partially drawn from the PhD dissertation of R.H.

---

## References


- Bogacki, P.; and Shampine, L. F. 1989. A 3(2) pair of Runge-Kutta formulas. *Applied Mathematics Letters* 2(4): 321–325.
- Brockman, G.; Cheung, V.; Pettersson, L.; Schneider, J.; Schulman, J.; Tang, J.; and Zaremba, W. 2016. OpenAI Gym. *arXiv preprint arXiv:1606.01540*.
- Che, Z.; Purushotham, S.; Cho, K.; Sontag, D.; and Liu, Y. 2018. Recurrent neural networks for multivariate time series with missing values. *Scientific Reports* 8(1): 1–12.
- Chen, T. Q.; Rubanova, Y.; Bettencourt, J.; and Duvenaud, D. K. 2018. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems*, 6571–6583.
- Cybenko, G. 1989. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems* 2(4): 303–314.
- Dormand, J. R.; and Prince, P. J. 1980. A family of embedded Runge-Kutta formulae. *Journal of Computational and Applied Mathematics* 6(1): 19–26.
- Dupont, E.; Doucet, A.; and Teh, Y. W. 2019. Augmented neural ODEs. In *Advances in Neural Information Processing Systems*, 3134–3144.
- Durkan, C.; Bekasov, A.; Murray, I.; and Papamakarios, G. 2019. Neural spline flows. In *Advances in Neural Information Processing Systems*, 7509–7520.
- Eldan, R.; and Shamir, O. 2016. The power of depth for feedforward neural networks. In *Conference on Learning Theory*, 907–940.
- Friston, K. J.; Harrison, L.; and Penny, W. 2003. Dynamic causal modelling. *NeuroImage* 19(4): 1273–1302.
- Funahashi, K.-I. 1989. On the approximate realization of continuous mappings by neural networks. *Neural Networks* 2(3): 183–192.
- Funahashi, K.-i.; and Nakamura, Y. 1993. Approximation of dynamical systems by continuous time recurrent neural networks. *Neural Networks* 6(6): 801–806.
- Gabrié, M.; Manoel, A.; Luneau, C.; Macris, N.; Krzakala, F.; Zdeborová, L.; et al. 2018. Entropy and mutual information in models of deep neural networks. In *Advances in Neural Information Processing Systems*, 1821–1831.
- Gholami, A.; Keutzer, K.; and Biros, G. 2019. ANODE: Unconditionally accurate memory-efficient gradients for neural ODEs. *arXiv preprint arXiv:1902.10298*.
- Gleeson, P.; Lung, D.; Grosu, R.; Hasani, R.; and Larson, S. D. 2018. c302: a multiscale framework for modelling the nervous system of *Caenorhabditis elegans*. *Phil. Trans. R. Soc. B* 373(1758): 20170379.
- Hanin, B.; and Rolnick, D. 2018. How to start training: The effect of initialization and architecture. In *Advances in Neural Information Processing Systems*, 571–581.
- Hanin, B.; and Rolnick, D. 2019. Complexity of linear regions in deep networks. *arXiv preprint arXiv:1901.09021*.
- Hanshu, Y.; Jiawei, D.; Vincent, T.; and Jiashi, F. 2020. On Robustness of Neural Ordinary Differential Equations. In *International Conference on Learning Representations*.
- Hasani, R. 2020. *Interpretable Recurrent Neural Networks in Continuous-time Control Environments.* PhD dissertation, Technische Universität Wien.
- Hasani, R.; Amini, A.; Lechner, M.; Naser, F.; Grosu, R.; and Rus, D. 2019. Response characterization for auditing cell dynamics in long short-term memory networks. In *2019 International Joint Conference on Neural Networks (IJCNN)*, 1–8. IEEE.
- Hasani, R.; Lechner, M.; Amini, A.; Rus, D.; and Grosu, R. 2020. The natural lottery ticket winner: Reinforcement learning with ordinary neural circuits. In *Proceedings of the 2020 International Conference on Machine Learning*. JMLR.org.
- Hochreiter, S.; and Schmidhuber, J. 1997. Long short-term memory. *Neural Computation* 9(8): 1735–1780.
- Holl, P.; Koltun, V.; and Thuerey, N. 2020. Learning to Control PDEs with Differentiable Physics. *arXiv preprint arXiv:2001.07457*.
- Hornik, K.; Stinchcombe, M.; and White, H. 1989. Multilayer feedforward networks are universal approximators. *Neural Networks* 2(5): 359–366.
- Hosea, M.; and Shampine, L. 1996. Analysis and implementation of TR-BDF2. *Applied Numerical Mathematics* 20(1-2): 21–37.
- Jia, J.; and Benson, A. R. 2019. Neural jump stochastic differential equations. In *Advances in Neural Information Processing Systems*, 9843–9854.
- Kingma, D. P.; and Ba, J. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
- Koch, C.; and Segev, K. 1998. *Methods in Neuronal Modeling — From Ions to Networks.* MIT Press, second edition.
- Lapicque, L. 1907. Recherches quantitatives sur l'excitation electrique des nerfs traitée comme une polarisation. *Journal de Physiologie et de Pathologie Générale* 9: 620–635.
- Lechner, M.; and Hasani, R. 2020. Learning Long-Term Dependencies in Irregularly-Sampled Time Series. *arXiv preprint arXiv:2006.04418*.
- Lechner, M.; Hasani, R.; Amini, A.; Henzinger, T. A.; Rus, D.; and Grosu, R. 2020a. Neural circuit policies enabling

auditable autonomy. *Nature Machine Intelligence* 2(10): 642–652.

- Lechner, M.; Hasani, R.; Rus, D.; and Grosu, R. 2020b. Gershgorin Loss Stabilizes the Recurrent Neural Network Compartment of an End-to-end Robot Learning Scheme. In *2020 International Conference on Robotics and Automation (ICRA)*. IEEE.
- Lechner, M.; Hasani, R.; Zimmer, M.; Henzinger, T. A.; and Grosu, R. 2019. Designing worm-inspired neural networks for interpretable robotic control. In *2019 International Conference on Robotics and Automation (ICRA)*, 87–94. IEEE.
- Lee, G.-H.; Alvarez-Melis, D.; and Jaakkola, T. S. 2019. Towards robust, locally linear deep networks. *arXiv preprint arXiv:1907.03207*.
- Montufar, G. F.; Pascanu, R.; Cho, K.; and Bengio, Y. 2014. On the number of linear regions of deep neural networks. In *Advances in Neural Information Processing Systems*, 2924–2932.
- Mozer, M. C.; Kazakov, D.; and Lindsey, R. V. 2017. Discrete Event, Continuous Time RNNs. *arXiv preprint arXiv:1710.04110*.
- Pascanu, R.; Mikolov, T.; and Bengio, Y. 2013. On the difficulty of training recurrent neural networks. In *International Conference on Machine Learning*, 1310–1318.
- Pascanu, R.; Montufar, G.; and Bengio, Y. 2013. On the number of response regions of deep feed forward networks with piece-wise linear activations. *arXiv preprint arXiv:1312.6098*.
- Penny, W.; Ghahramani, Z.; and Friston, K. 2005. Bilinear dynamical systems. *Philosophical Transactions of the Royal Society B: Biological Sciences* 360(1457): 983–993.
- Pontryagin, L. S. 2018. *Mathematical Theory of Optimal Processes.* Routledge.
- Poole, B.; Lahiri, S.; Raghu, M.; Sohl-Dickstein, J.; and Ganguli, S. 2016. Exponential expressivity in deep neural networks through transient chaos. In *Advances in Neural Information Processing Systems*, 3360–3368.
- Press, W. H.; Teukolsky, S. A.; Vetterling, W. T.; and Flannery, B. P. 2007. *Numerical Recipes 3rd Edition: The Art of Scientific Computing.* Cambridge University Press.
- Quaglino, A.; Gallieri, M.; Masci, J.; and Koutník, J. 2020. SNODE: Spectral Discretization of Neural ODEs for System Identification. In *International Conference on Learning Representations*.
- Raghu, M.; Poole, B.; Kleinberg, J.; Ganguli, S.; and Dickstein, J. S. 2017. On the expressive power of deep neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, Volume 70, 2847–2854. JMLR.org.
- Rubanova, Y.; Chen, R. T.; and Duvenaud, D. 2019. Latent ODEs for irregularly-sampled time series. *arXiv preprint arXiv:1907.03907*.
- Rumelhart, D. E.; Hinton, G. E.; and Williams, R. J. 1986. Learning representations by back-propagating errors. *Nature* 323(6088): 533–536.
- Sarma, G. P.; Lee, C. W.; Portegys, T.; Ghayoomie, V.; Jacobs, T.; Alicea, B.; Cantarelli, M.; Currie, M.; Gerkin, R. C.; Gingell, S.; et al. 2018. OpenWorm: overview and recent advances in integrative biological simulation of *Caenorhabditis elegans*. *Phil. Trans. R. Soc. B* 373(1758): 20170382.
- Schölkopf, B. 2019. Causality for Machine Learning. *arXiv preprint arXiv:1911.10500*.
- Serra, T.; Tjandraatmadja, C.; and Ramalingam, S. 2017. Bounding and counting linear regions of deep neural networks. *arXiv preprint arXiv:1711.02114*.
- Shampine, L. F. 1975. *Computer Solution of Ordinary Differential Equations. The Initial Value Problem.*
- Todorov, E.; Erez, T.; and Tassa, Y. 2012. MuJoCo: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 5026–5033. IEEE.
- Wicks, S. R.; Roehrig, C. J.; and Rankin, C. H. 1996. A dynamic network simulation of the nematode tap withdrawal circuit: predictions concerning synaptic function using behavioral criteria. *Journal of Neuroscience* 16(12): 4017–4031.
- Zhuang, J.; Dvornek, N.; Li, X.; Tatikonda, S.; Papademetris, X.; and Duncan, J. 2020. Adaptive Checkpoint Adjoint Method for Gradient Estimation in Neural ODE. In *Proceedings of the 37th International Conference on Machine Learning*. PMLR 119.